



Project Proposal — Constituency Intelligence Engine

FOR REVIEW

working title · name TBD · Track 1 — "Build with AI: Code for Communities" (Google Cloud)

Prepared for: Tech Lead review & approval to proceed · **Prepared by:** [Your name] · **Date:** 23 June 2026
Track: People's Priorities — AI for Constituency Development Planning · **Demo target:** Dharmapuri Lok Sabha, Tamil Nadu
Submission deadline: 8 July 2026 · **Demo Day:** 23 July 2026, New Delhi

1. Executive summary

We are building a **Constituency Intelligence Engine** (*working title — name TBD*) — an AI decision-support system for Members of Parliament that turns scattered, multilingual citizen demands and fragmented public data into a **clear, defensible action plan**.

The differentiating insight: an MP's own fund (**MPLADS $\approx$ ₹5 crore/year**) is tiny next to the **hundreds of crores of existing government schemes** (housing, water, jobs, pensions) already meant for their constituents. **Most citizen needs are unfilled entitlements, not requests for new money**. So instead of "yet another complaint dashboard," our engine:

1. Routes each citizen need to the **scheme that should already cover it**.
2. Shows, village-by-village, **where scheme money is failing to land** (the "Constituency X-Ray").
3. Recommends **(A)** unlock existing entitlements for ₹0 of the MP's fund, and **(B)** spend the ₹5 cr only on the true gaps nothing else covers — fairly, including villages that never complain.

Status: concept finalised, design spec written, **clickable HTML prototype already built**, intake approach decided. **We are seeking approval to begin the 15-day build.**

2. The problem (Track 1, as stated)

MPs receive development requests through public meetings, letters, social media, grievance portals, and direct representations — while local development plans contain dozens of competing proposed projects. There's no objective way to consolidate citizen feedback, spot recurring needs, and weigh competing proposals against real demand.

Our sharper framing: decisions favour whoever **complains loudest**; they are ad-hoc and politically exposed; **silent, poor, remote areas are systematically ignored**; and the MP's scarce fund is often spent on things **other schemes should already cover** while genuine entitlement-delivery failures go unaddressed.

3. Our solution

"We don't help an MP spend ₹5 crore. We help them unlock the ₹500 crore of schemes already meant for their people — and spend their own ₹5 crore only on the gaps nothing else covers."

The engine — a two-floor model

We build **both floors**. Floor 1 is exactly what the problem statement asks for (done solidly for full Problem-Solution Fit); Floor 2 is our differentiator, built **on top of** Floor 1 — not instead of it.

 Floor 1 — the required basics (consolidate → theme → map → rank):

- 1. **Analyze submissions** — Tamil/English voice, text & photo → structured demand record, auto-categorised (water/road/school/health) via Gemini + Speech-to-Text + Translation.
- 2. **Detect recurring themes** — cluster submissions into recurring needs per ward by volume + recency ("drinking water = 400 requests, biggest issue").
- 3. **Map demand hotspots** — Google Maps view of where complaints concentrate (Ward 3 = red, 250 road complaints).
- 4. **Combine public data + rank** — fuse demand with Census/NFHS/UDISE + infrastructure gaps + the local development plan (RAG over the real plan, not invented projects) → a transparent ranked project list with "why this rank" reasoning.

 Floor 2 — our differentiator (built on top):

- 5. **Match to scheme — "whose money is this?"** — route each need to the responsible scheme/fund (housing → PMAY-G) vs. "no scheme covers this → MPLADS candidate."
- 6. **Gap math + two tracks** — gap = eligible – already covered; Track A unlock entitlements (₹0) + Track B spend ₹5 cr only on true gaps, each auto-justified.
- 7. **Silent-village detection** — high measured need but zero petitions.
- 8. **Anti-lobbying guard** — detect coordinated message floods and down-weight them.
- 9. **Close the loop** — citizen status updates → trust + repeat usage.

*Design principle: ranking is a **transparent weighted formula** (Demand × Need × Feasibility + Equity-boost), not a black box — government only deploys what it can explain and audit.*

Why this is differentiated (honest)

Most teams stop at Floor 1 (collect + categorise + heatmap + rank). **We do Floor 1 properly AND add the Floor 2 scheme-convergence layer** — requiring real understanding of how Indian governance/money works and integrating real scheme-coverage data. **The moat is domain insight + data execution, not a single feature.** We deliberately **avoid fabricated "impact prediction" numbers** (e.g. "reduces dropout by 12%") — unprovable and a knowledgeable judge/MP will reject them. All our figures are **computable and real** (people served, eligible-but-unserved counts, distance to nearest facility).

4. Key features (build scope)

Floor	Feature	Notes
1	Multilingual (Tamil) voice/text/photo intake	Speech-to-Text + Translation + Gemini
1	Recurring-theme detection	Embeddings clustering by volume + recency
1	Demand hotspot map	Google Maps — where complaints concentrate
1	Transparent ranked project list ("why this rank")	Demand × Need × Feasibility + Equity

1	RAG over local development plan	Real proposed projects, not invented
2	Scheme-matching & entitlement routing	The core differentiator
2	Constituency X-Ray (village coverage heatmap)	Hero screen, Maps Platform
2	Two-track recommendations (Unlock / Spend ₹5 cr)	OR-Tools budget optimisation
2	Silent-village (equity) detection	Need-weighted, not complaint-weighted
2	Anti-lobbying / astroturfing guard	Down-weight coordinated floods
2	Auto-drafted official letters & memos	Gemini
2	Close-the-loop citizen status updates	Trust + repeat usage
2	Satellite "proof" for 2–3 hero villages	Earth Engine (depth, not breadth)
2	(Stretch) Voice "ask your constituency" assistant	Gemini over BigQuery — demo flourish

5. Intake approach

Both feed the **same backend pipeline** (no duplicate work):

- **Tier 1 — own web + voice page (reliable backbone):** Tamil mic/upload; DB pre-loaded with 200–500 synthesised realistic Tamil petitions mapped to real Dharmapuri villages. 100% under our control.
- **Tier 2 — Telegram bot (live "wow"):** free, no approval, accepts Tamil voice notes; a judge can message it live and see the demand appear on the dashboard in seconds.
- **WhatsApp: out of scope** (Business API approval too slow). Pitch line: "intake is channel-agnostic; built to plug into WhatsApp / CM Helpline in production."

6. Architecture (Google Cloud native)

```

Citizen (Tier 1 web voice / Tier 2 Telegram voice)
  |
  v
Cloud Function / Cloud Run webhook (intake)
  |
  Speech-to-Text (Tamil) -> Translation API
  |
  Gemini (Vertex AI): extract need + scheme match -> structured JSON
  |
  v
BigQuery -- demand records + public datasets
  (Census/UDISE/NFHS/PMAY-G/JJM/MGNREGA), joined on LGD codes
  |
  |   BigQuery GIS joins + transparent Need/Gap score
  |   + OR-Tools budget optimisation
  v       v
Cloud Run API (FastAPI) -> Next.js dashboard (Google Maps Platform)
                        + Gemini-drafted letters
                        + Earth Engine imagery (hero villages)

```

Stack: Next.js · FastAPI on Cloud Run · Cloud Functions · Vertex AI / Gemini · Cloud Speech-to-Text · Translation API · BigQuery + BigQuery GIS · OR-Tools · Earth Engine · Google Maps Platform · Firebase. All serverless; runs on hackathon Google Cloud credits.

7. Data plan (real Tamil Nadu sources)

Layer	Source	Resolution
Geographic join key	LGD (Local Government Directory)	village/ward
Demographics & amenities	Census 2011 (TN)	village
Education need	UDISE+ (TN)	school/village
Health need	NFHS-5 (TN) + PHC data	district
Housing coverage	PMAY-G MIS / AwaasSoft	village/block
Water coverage	Jal Jeevan "Har Ghar Jal" dashboard	village
Jobs/assets	MGNREGA MIS	panchayat
Anti-duplication	MPLADS portal (filter TN MP)	work-level
Satellite proof	Earth Engine / Bhuvan	image
Citizen petitions	Synthesised (real feeds not open)	—

Honesty notes (stated in pitch): petitions are synthesised; some scheme data is block/district-level only (best-available used); satellite verification deep for 2–3 hero villages, simulated for breadth.

8. Scope for the hackathon

In: one constituency (Dharmapuri); real Census + UDISE + 3–4 scheme datasets (PMAY-G, Jal Jeevan, MGNREGA + one pension/PM-Kisan); Tamil intake (Tier 1 + Tier 2); scheme-matching; X-Ray; transparent scoring; two-track recommendations; auto-letters; ₹5 cr optimiser; silent detection; 2–3 satellite hero villages.

Out (stated honestly): all 30+ schemes; live WhatsApp; statewide rollout; causal outcome/impact prediction.

8a. Submission deliverables (exactly what the hackathon requires)

The hackathon requires **four** items — we will deliver all four:

1. **Source code** — public / access-granted GitHub repository.
2. **Demo video (3–5 min)** — product working end-to-end.
3. **Pitch deck (10–12 slides)** — problem · solution · AI/technical approach · who it serves · why deployable · how it scales.
4. **Deployed prototype link** — live on Cloud Run / Firebase.

*Rule from the brief: the prototype **must** utilise **Google Cloud technologies** and show a **functioning end-to-end flow**. Our architecture (§6) is GCP-native and end-to-end by design.*

8b. Requirements coverage — nothing missed

What the hackathon asks	Where we cover it
Citizens submit via voice, text, photos, messaging apps	Floor 1 intake (Tamil voice/text/photo) + Tier 2 Telegram; WhatsApp-ready
Analyze submissions	Floor 1 step 1 (categorise)
Surface recurring themes	Floor 1 step 2 (theme clustering)
Map demand hotspots	Floor 1 step 3 (Google Maps hotspot view)
Combine with demographic data, infrastructure gaps, local development plans, public datasets	Floor 1 step 4 (Census/NFHS/UDISE + infra + RAG over local development plan)
Recommend & rank high-priority works	Floor 1 step 4 (ranked list) + Floor 2 (two-track)
Multilingual / low-literacy / low-connectivity	Tamil voice-first; SMS/IVR fallback; Inclusivity 15%
Use Google Cloud	Entire stack is GCP-native (§6)
Recommended tools (Gemini, Vertex AI, Speech-to-Text, Translation, Maps, BigQuery, Firebase, Cloud Run, Earth Engine)	All used (§6)
Deployable in weeks; scalable	Serverless; 1 MP → 39 MPs + 234 MLAs; rollout §10
Non-technical MP gets value in 5 min	Auto-justification memos + clean dashboard

9. Indicative 15-day plan

Days	Focus
1–3	Data acquisition + cleaning (LGD, Census, UDISE, scheme dashboards); BigQuery schema
4–6	Intake pipeline (Tier 1 web + Tier 2 Telegram) → Speech-to-Text → Gemini → BigQuery
5–8	Scheme-matching + Need/Gap scoring + silent detection (hardest data-join work — most time here)
7–10	Dashboard: X-Ray map + scheme-gap list + two-track screens
9–12	OR-Tools budget optimiser + auto-letters/memos; deploy to Cloud Run
12–13	Satellite hero villages + (stretch) voice assistant
13–14	Demo video, pitch deck, polish
15 (8 Jul)	Submit (source code, demo video, pitch deck, deployed link)

10. Production rollout (the "what's next?" answer)

- **Phase 1 — Pilot (90 days, one constituency):** data-sharing MoU with district admin; connect one real intake channel; office staff validate flagged gaps; measure petitions processed, ₹ entitlements unlocked, forgotten villages reached.
- **Phase 2 — Harden (3–6 mo):** privacy & security to government standard (consent, minimal PII, MeitY-empanelled/compliant cloud, GIGW); integrate CM Helpline 1100 / e-Sevai / CPGRAMS.
- **Phase 3 — Scale (6–18 mo):** roll out to TN's 39 MPs + 234 MLAs, then other states; natural owner = state e-gov agency (TNeGA) / District Planning Committee. Serverless ⇒ low run cost; scaling is configuration, not rebuild.

11. Risks & mitigations

Risk	Mitigation
Concept is graspable once described	Win on execution depth with real TN data; don't over-explain pre-submission
Hardest work = messy data joins (Tamil → LGD → scheme coverage)	Front-load it (days 1–8); most effort here, not UI
Scheme data resolution varies	Use best-available level; be transparent
"This is just convergence/DPC planning"	Rebuttal: "Yes — it fails because nobody joins the data at village level; we did."
Scope creep in 15 days	Strict in/out scope; depth over breadth

12. Team & resources needed

- **Team strengths:** frontend, backend/APIs, AI/ML & data, mobile/design (full-stack).
- **Lead:** Google Cloud Professional Architect (deployment & GCP services).
- **Needed:** Google Cloud credits (provided to shortlisted teams), Vertex AI / Gemini access, Maps Platform key, Telegram bot token (free).

13. Approval requested

We are asking the Tech Lead to approve:

- ☐ **1. Direction — proceed with this Constituency Intelligence Engine (scheme-convergence / entitlement-gap engine) as the Track 1 entry.**
- ☐ **2. Tech stack — Google Cloud native stack as in §6.**
- ☐ **3. Scope — the in/out scope in §8 (depth over breadth, no fabricated impact prediction).**
- ☐ **4. Intake — Tier 1 (own web/voice) + Tier 2 (Telegram bot).**
- ☐ **5. Go-ahead to start the detailed 15-day build.**

References in repo: [prototype/index.html](#) · [docs/track1-design-spec.md](#) · [docs/build-with-ai-code-for-communities.md](#)

Reviewer comments: _____

Decision: ☐ Approved ☐ Approved with changes ☐ Needs rework

Signature / Date: _____